DMSim: Trace-Driven Simulation of Differentiated Memory for AWS Trainium2 LLM Decode

Arina Shah
Stanford University
arinas@stanford.edu

Michael Tsien
Stanford University
tsienmd@stanford.edu

Thierry Tambe
Stanford University
ttambe@stanford.edu

***Abstract*—The AWS Trainium2 is a hardware accelerator built for distributed machine learning workloads, including large language model (LLM) inference. Despite its large HBM pool, Trainium2 remains subject to memory bandwidth pressure during autoregressive decode. Leveraging heterogeneous differentiated memory systems has emerged as a promising solution to address this issue. We present DMSim, a trace-driven analytical simulator that evaluates the Trainium2 memory hierarchy of PSUM, SBUF, and HBM, expanded to include short-term RAM (StRAM) and long-term RAM (LtRAM) under constant die-area constraints. Using AWS Neuron Explorer profiles of Llama-3.2-1B-Instruct and Qwen1.5-MoE-A2.7B decode on a trn2.3xlarge instance, we compare baseline Trainium2 against various memory hierarchy configurations and placement policies, including a full tiered policy that places weights in LtRAM and KV cache and activation tensors, including hidden states, in StRAM. On Llama, the best full hierarchy design (50% SBUF to StRAM, 25% HBM to LtRAM) reduced HBM traffic by 28.6%. On Qwen, capacity-heavy LtRAM allocation (25% SBUF to StRAM, 50% HBM to LtRAM) achieved the highest HBM reduction of 39.6%. Our results show that differentiated heterogeneous memory benefits depend strongly on model weight footprint and that iso-area hierarchy design must match workload characteristics. We plan to open source the DMSim infrastructure for the research and developer communities.**

***Index Terms*—differentiated memory, AWS Trainium2, HBM, eDRAM, RRAM, FeRAM, mixture of experts, simulation**

I. Introduction

Autoregressive LLM decode is memory-bound: each generated token triggers repeated reads of model weights and a growing key-value (KV) cache, dominating inference latency [1]. On AWS Trainium2, seen in Fig. 1, compute sits behind a hierarchy of on-chip scratch buffers (PSUM, SBUF) and a large HBM pool [2], [3]. While the large HBM memory pool reduces the stress of the memory bandwidth wall, Trainium2 remains subject to memory bandwidth pressure.

In the Trainium2 architecture, the strict use of SRAM and HBM limits flexibility in memory optimization. Tensors with distinct lifetimes and access frequencies, such as persistent weights and KV cache, are treated similarly and placed in the same HBM pool. Higher-density or lower-energy memory technologies could be better matched to these differing access patterns, but the fixed hierarchy leaves no mechanism to exploit them.

To address this data movement problem, we explore heterogeneous differentiated memory hierarchies using emerging memory technologies that can be classified as dense short-term

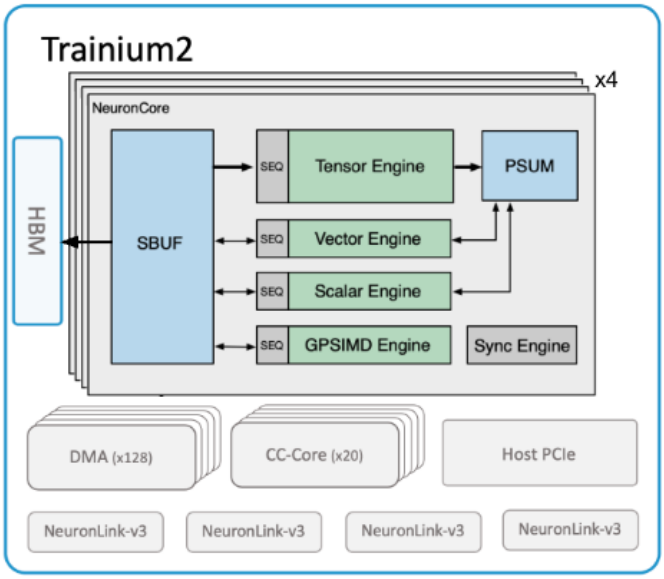


Fig. 1: AWS Trainium2 device architecture diagram with detailed NeuronCore view based on AWS Neuron Documentation [2], [4]. Key blocks in this diagram are the HBM on-device memory and NeuronCore-v3 compute acceleration units. Trainium2 instances have multiple configurable NeuronCore counts. In this paper, we use trn2.3xlarge, which has four logical NeuronCore-v3s.

RAM (StRAM), such as 1T1C and 3T gain-cell (GC) embedded DRAM (eDRAM), and long-term RAM (LtRAM), such as resistive RAM (RRAM) and ferroelectric RAM (FeRAM) [5]. These technologies' varying characteristics, such as density, latency, or energy, can be matched to tensor data lifetimes, using StRAM for latency-sensitive, transient working sets and LtRAM for persistent, read-intensive data like weights. However, their usage and effects on metrics like power and latency are application-specific [5].

In this work, we present DMSim, a trace replay simulator for the AWS Trainium2 accelerator that models placement, residency, transfer cost, and refresh energy without re-executing models on silicon or cycle-accurate hardware simulations. The simulator assumes iso-area conditions, trading SBUF area for StRAM and HBM area for LtRAM, mapping these technologies to existing Trainium2 memory tiers without requiring

changes to die footprint. DMSim employs semantic tensor classification to enable configurable tiered placement policies that route each tensor class to the desired memory technology, allowing for exploration of various memory hierarchies and their effects. Concretely, this paper addresses three questions: (1) How much traffic between on-chip and off-chip memory, referred to as cross-domain traffic, can tiered placement remove under iso-area budgets? (2) Which technology (1T1C vs. 3T GC; RRAM vs. FeRAM) and area split is best? (3) Do sparse mixture-of-experts (MoE) models behave differently from dense models under such differentiated heterogeneous memory compositions?

The proposed differentiated hierarchies yield substantial improvements in metrics like cross-domain traffic and energy. Under iso-area constraints, simulated full tiered hierarchy configurations reduce HBM traffic by up to 28.6% on the dense model and 39.6% on the sparse MoE model. StRAM-only additions (1T1C) achieved 13.6% cross-domain traffic reduction for the dense model, while LtRAM-only additions (RRAM) yielded a 43.8% energy reduction. Combined StRAM and LtRAM configurations further reduced cross-domain traffic by up to 12.1%. Critically, optimal hierarchy design is workload-dependent: KV-cache-heavy dense models benefit most from strong on-chip StRAM capacity, while weight-heavy MoE models require larger LtRAM pools to capture their dominant traffic component. The primary contributions of this paper are:

- DMSim, a trace-driven simulator that models tiered placement, tensor residency, transfer cost, and refresh energy for heterogeneous memory hierarchies on AWS Trainium2, without requiring cycle-accurate simulation or hardware modification (Section III).
- A semantic tensor classification methodology that maps compiler-level Neuron Explorer DMA traces to weight, KV cache, activation, and other categories, enabling placement policies specifically matched to tensor access patterns (Section III).
- An iso-area hierarchy evaluation across StRAM and LtRAM technologies and area split design points, characterizing cross-domain traffic and energy tradeoffs for both dense and sparse MoE LLMs (Section V).

The remainder of this paper is organized as follows. Section II describes the Trainium2 memory hierarchy and baseline decode profiles. Section III presents the DMSim design. Section IV details the experimental setup. Section V describes the evaluation and results. Section VI discusses limitations and future work, and Section VII concludes the paper.

II. Background and Baseline

A. Trainium2 Memory Hierarchy

The Trainium2 accelerator organizes memory in a traditional tiered memory hierarchy (Fig. 2). Nearest to compute on each NeuronCore-v3 are the on-chip SRAM buffers, the PSUM scratchpad (2 MiB) and the SBUF (28 MiB). PSUM accumulates matrix multiplication partial sums, while SBUF is used to store input and output activations. Additionally, the

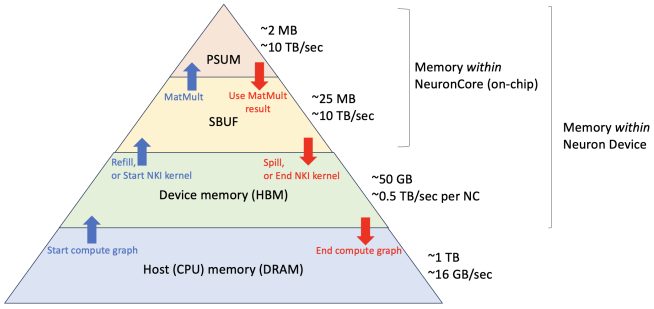


Fig. 2: AWS Trainium2 memory hierarchy from the AWS Neuron Documentation. The architecture has four tiers: PSUM, SBUF, HBM, and DRAM. DRAM is host memory and considered off-device. HBM is device memory and considered on-device but off-chip, and PSUM and SBUF are the on-device, on-chip SRAM scratchpad buffers [3]. Our work considers the on-device memories. Additionally, a key parameter for our work is the bandwidth between SBUF and HBM, used to characterize cross-domain bandwidth at 368 GB/s [14].

SBUF and PSUM states are wiped at kernel boundaries. The next-lowest tier is HBM (96 GiB), which is considered on-device and shared between the four logical NeuronCores. The lowest level in the Trainium2 hierarchy is the DRAM for the host CPU, which is not considered on-device [2]–[4]. Our baseline simulator condition models the on-device three-level stack of PSUM, SBUF, and HBM, with all persistent tensors homed in HBM.

In our proposed differentiated memory hierarchy, we evaluate adding StRAM and LtRAM memories to the existing hierarchy. We characterize 1T1C and 3T GC eDRAM as StRAM candidates using density, latency, and refresh parameters drawn from the literature. For LtRAM, we evaluate RRAM and FeRAM. Capacities are derived from iso-area trades with SBUF or HBM and bit-cell density metrics.

B. Baseline Measurements

To establish baselines, we profile LLM decode for Llama-3.2-1B-Instruct and Qwen1.5-MoE-A2.7B on a trn2.3xlarge instance using the AWS Neuron Explorer tool [15]–[19]. Table II summarizes baseline simulation results, gathered using the methods described in Section III. It is critical to note that the time metric presented in Table II only accounts for the accumulated memory access time. Furthermore, while the Trainium2 device memory includes PSUM, SBUF, and HBM, only traffic between SBUF and HBM is characterized. The Neuron Explorer tool captures memory traffic through DMAs, which lie on the cross-domain interface between on-chip and off-chip. PSUM does not interact with the DMAs, so the traces do not account for PSUM traffic. However, the bandwidth of the PSUM is very high compared to the cross-domain bandwidth shown in Fig. 2 [3], meaning that accesses have minimal latency. Similarly, the size of the PSUM at 2 MiB is

TABLE I: Technology parameters for memories configured in DMSim. Values are drawn from existing literature and CACTI profiling for SBUF [6].

Technology	Read / Write Latency (ns)	Read / Write Energy (pJ/bit)	Refresh Interval	Refresh Energy (pJ/bit)	Cell Density (bits/μm^2)
1T1C StRAM [7], [8]	1.5 / 1.5	0.5 / 0.5	90 μs	0.5	34.5
3T GC StRAM [9]	2.5 / 2.5	0.04 / 0.04	77 μs	0.021	12.5
RRAM [10], [11]	10 / 10	0.5 / 1.0	—	—	62.5
FeRAM [10], [11]	10 / 10	0.5 / 0.1	—	—	30.0
SBUF [6]	0.8 / 0.8	0.01 / 0.03	—	—	2.44
HBM [12], [13]	122 / 122	3.9 / 3.9	32 ms	7.8	961.5

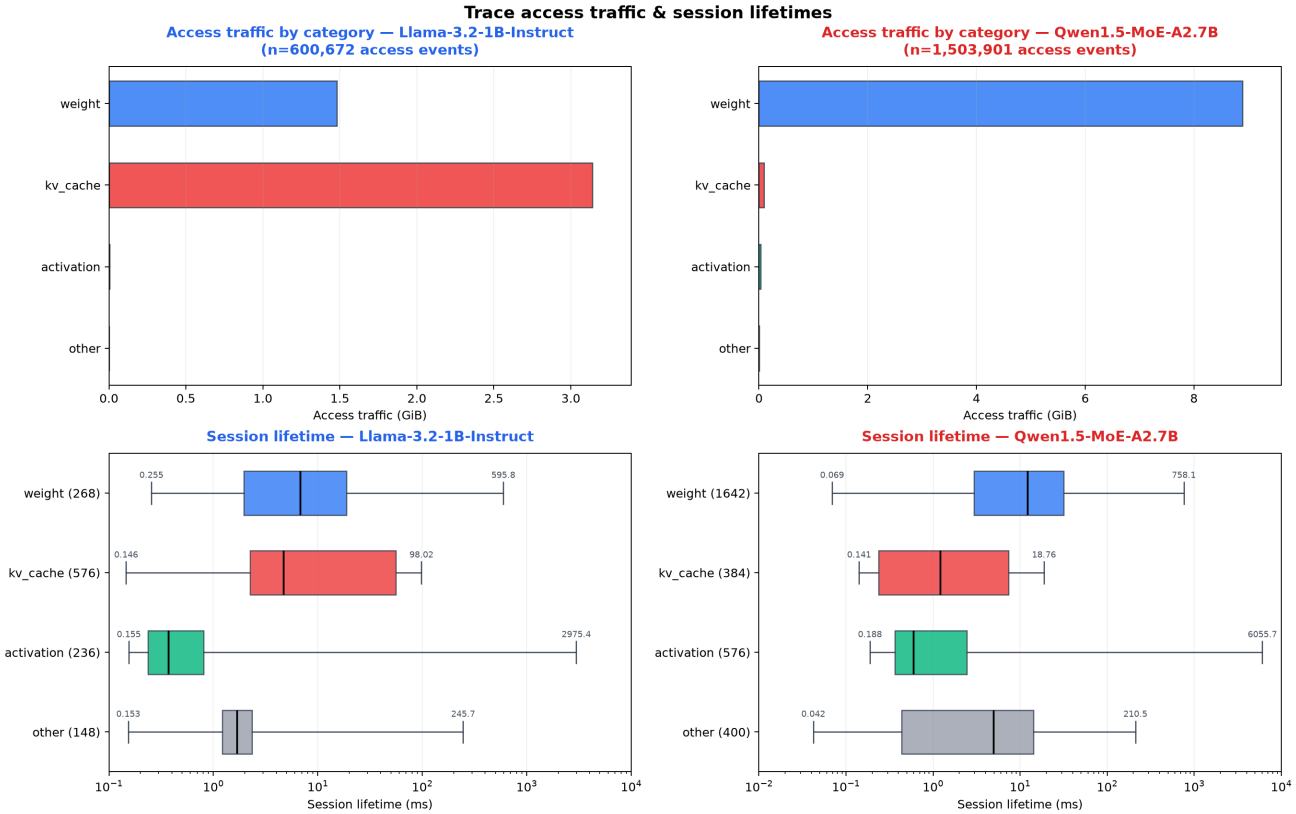


Fig. 3: Access traffic and session lifetime per tensor category for Llama and Qwen decode stage traces. The plots on the left for the dense Llama model exhibit a computational emphasis on the KV cache. For the sparse Qwen MoE model on the right, the emphasis lies with weight tensors.

small compared to the other memories, meaning that it likely constitutes a relatively small portion of the overall energy.

From the baseline profile, we determined distinct tensor categories of weight, kv_cache, activation, and other based on trace execution order and shapes. Fig. 3 reveals that KV cache dominates traffic in Llama while weights dominate in Qwen, a distinction that motivates the different placement strategies explored. As expected, KV cache accesses dominate the access traffic for the dense Llama model as the KV cache is repeatedly accessed and grows with each generated token. For Qwen, weights dominate the traffic, as MoE models activate new experts for each token and must read in the associated weights [1]. These baselines anchor all percentage improvements reported in Section V.

The Qwen profile exhibits roughly 30% more HBM (and therefore cross-domain) traffic and about two times more HBM to SBUF hops, reflecting a larger MoE weight footprint and working set (Table II). Regarding the tensor lifetimes in Fig. 3, while time resident in HBM would be the preferred metric, the trace DMA traffic cannot fully represent the HBM residence. Instead, the lifetime analysis uses the time from the first access to the last access of a tensor. Through this analytical lens, the

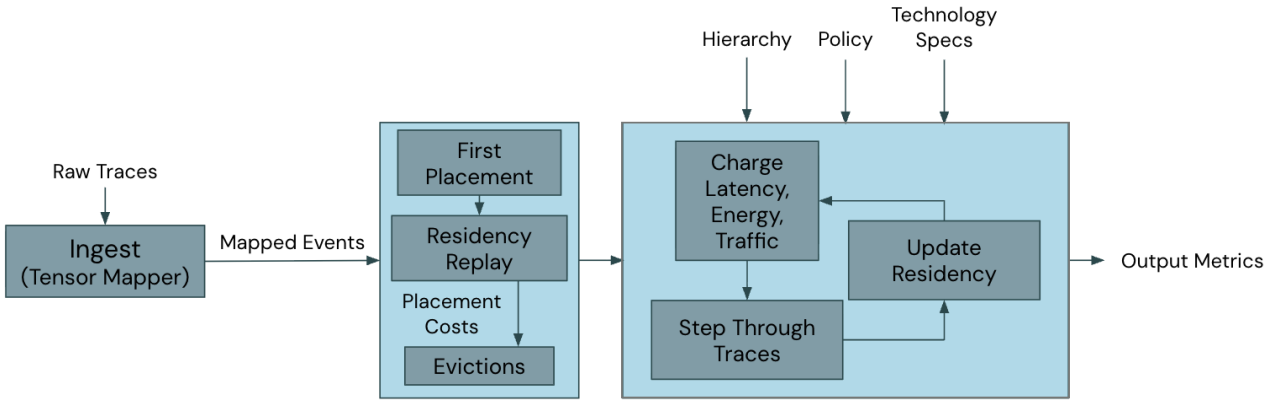


Fig. 4: DMSim simulation flow. DMSim first ingests the JSON output traces from Neuron Explorer and maps them to tensor types. Then, it performs rough placement by placing tensors in their memory tier homes and replays the trace to calculate retention scores and determine spillage. After, the simulator can step through the traces with the given hierarchy, policy, and technology configurations to determine latency, energy, and cross-domain traffic.

TABLE II: Baseline decode profile with four logical NeuronCores and all tensors homed in HBM. Results are baseline DMSim outputs.

Model	Memory Access Time (ms)	HBM Traffic (MB)	HBM to SBUF Hops	Total Energy (J)
Llama-3.2-1B-Instruct	4.49	26.16	4070	17.71
Qwen1.5-MoE-A2.7B	6.51	34.14	8409	385.9

effect of the model architectures can be seen in the lifetimes. For Llama, the maximum lifetime is represented by the KV cache, aligning with the KV-heavy access pattern of the model. Similarly, the weight lifetimes tend to be longer than the other tensors for Qwen, reflecting its weight-heavy access pattern.

Baseline energy in Table II is dominated by HBM refresh energy rather than access energy. DMSim baseline access energy across all DMA events is only 1.3 mJ for Llama and 2.0 mJ for Qwen. The remainder of the reported total energy is HBM refresh energy, which accumulates over the full profile span for all bytes allocated in HBM between trace events, even though the summed memory access latency in Table II is a few milliseconds. Thus, total energy is mostly informative for relative comparisons across hierarchy configurations, not as an absolute measure of per-decode energy cost. Because access energy is negligible in all configurations, subsequent experiments report only total energy, which primarily reflect the refresh characteristics of whichever volatile memory tiers are occupied.

III. DMSim Design

DMSim ingests Neuron Explorer execution traces from a Trainium2 device and replays them against a configurable memory hierarchy, accumulating latency, energy, and cross-domain traffic metrics without re-executing the model on silicon. Three design decisions shape the simulator: how traces are replayed, how tensors are classified for placement, and how memory capacities are derived under iso-area constraints.

A. Trace-Driven Analytical Replay

Instead of cycle simulation or re-running models, DMSim replays timestamped memory access events extracted from Neuron Explorer profiles. Each event specifies a tensor, an operation, a byte count, a target level, and the NeuronCore. The simulator accumulates worst-core latency, per-level and total energy, HBM byte traffic, and cross-domain traffic. The data flow, shown in Fig. 4, is as follows:

1) Inputs: Raw Neuron Explorer JSON profile traces are ingested, combined across all cores, then mapped to tensor classes.
2) Configurations: The hierarchy YAML defines the memory tiers, interconnect bandwidths, and area trades, the policy YAML describes fallback and home tiers, and the tech_spec configuration YAMLs detail the memory specifications in Table I.
3) Placement: Assign tensor homes by tensor classification (weight, kv_cache, activation, other), spilling overflow to the fallbacks defined in the policy configuration.
4) Replay: For each access or kernel boundary event, update tensor residencies and accumulate latency, energy, and traffic.
5) Output: Summarize metrics (e.g. latency, energy, refresh, HBM traffic) in a results JSON.

B. Semantic Tensor Classification for Tiered Placement

While the raw traces and DMA records have names to classify tensors, they are not used to identify weights or KV cache tensors. We built a NeffTensorCatalog that maps NEFF trace metadata and DMA heuristics to semantic categories, enabling policies such as “weights placed in LtRAM, KV cache placed

in StRAM." The categories used for tensor classification are weight, kv_cache, activation, and other. The access patterns of these classes are used to strategically determine placement. We evaluate four policies:

- Baseline: all tensor classifications are homed in HBM
- StRAM-only: kv_cache tensors homed in StRAM; all other tensors in HBM
- LtRAM-only: weight tensors homed in LtRAM; all other tensors in HBM
- Tiered: weight tensors homed in LtRAM; kv_cache and activation in StRAM; other in HBM

Placement runs once at simulation start. Specifically, a rough placement is performed first, placing tensors into their desired home tier. Then, the simulator constrains the memory levels, and tensors exceeding their tier capacity spill to the tier's fallback level defined in the configured policy. The default fallback tier is HBM. To determine the tensors to be spilled to the fallback level, DMSim performs a replay to find retention scores. For each tensor access that would incur a cross-domain read hop, the tensor accumulates a retention score of the difference between hop access latencies from the fallback and the desired home levels to the target level. Scores accumulate across all cross-interconnect read accesses to that tensor, so tensors frequently read from off-chip gain proportionally higher scores. Once the retention scores are calculated, tensors are spilled to the fallback level. We define HBM as the fallback level for all tested policies. The tensors with the highest retention benefit score are kept in the desired home tier.

C. Tier-Aware, Iso-Area Cost Accounting

The hierarchy configurations in the DMSim Trainium2 simulator enable five levels: PSUM, SBUF, StRAM, LtRAM, HBM. However, as discussed in Section II, since PSUM is not accessed through DMA, it is disregarded in the simulator. Configuration YAML files specify per-level technology specifications, DMA vs. on-chip bandwidth, and area-trade fractions.

The decision to trade SBUF area for StRAM and HBM area for LtRAM stems from the typical applications of the StRAM and LtRAM memory classes: StRAM to store the frequently accessed, transient tensors, and LtRAM to store read-intensive, infrequently written tensors [5]. The frequent access and transience of StRAM-preferred tensors make the SBUF scratchpad a suitable target for StRAM replacement. For LtRAM, the frequent reads and infrequent writes of weight tensors make them a fitting candidate for residency, and their shared use across all NeuronCores made the shared HBM the preferred target for LtRAM replacement.

TABLE III: Full Hierarchy Design Points

Design	Replaced SBUF Area	Replaced HBM Area
Primary	50%	25%
Balanced	25%	25%
Near-core Heavy	50%	10%
Capacity Heavy	25%	50%

Table III lists the four iso-area design points studied for the full hierarchy (when StRAM and LtRAM exist simultaneously). The 50% SBUF and 25% HBM split for the Primary hierarchy configuration was chosen to reflect a realistic yet aggressive iso-area trade. Because SBUF is smaller than HBM in capacity, a 50% trade yields a modest StRAM capacity. Whereas for LtRAM, 25% HBM replacement provides enough capacity to produce measurable HBM traffic reduction. The remaining three design points (Table III) then vary each dimension independently to isolate the contribution of StRAM and LtRAM area to the overall traffic reduction. Capacities follow constant die-area formulas:

$$C_{new,bytes} = f \cdot C_{old,bytes} \cdot \frac{\rho_{new,bits}}{\rho_{old,bits}} \tag{1}$$

$C_{new,bytes}$ and $C_{old,bytes}$ are the capacities of the StRAM or LtRAM replaced area and SBUF and HBM area, respectively. f is the fraction of the total SBUF or HBM area to be replaced, and $\rho_{new,bits}$ is the density of bits per square micron of StRAM and LtRAM. $\rho_{old,bits}$ is that density for the SBUF and HBM.

As an example, for the Primary design point (Table III) with 50% SBUF replacement using 1T1C eDRAM:

$$C_{1T1C,new} = \frac{1}{2} \cdot 28 \text{ MiB} \cdot \frac{34.5}{2.44} = 198.0 \text{ MiB} \tag{2}$$

D. Cost Model

Latency: To determine latency, there are two unique cases. The first is a hop between a source and a target tier. This latency is determined by the following equation:

$$t_{hop} = t_{read} + t_{write} + \frac{n_{bytes}}{BW_{link}} \tag{3}$$

where t_{read} is the read access time of the source and t_{write} is the write access latency of the target, both in seconds. n_{bytes} is the number of bytes to be transferred, and BW_{link} is the bandwidth of the connection in GB/s, defined in the hierarchy. For our Trainium2 simulator, we assume that traffic between on-chip and off-chip is limited by the DMA bandwidth of 368 GB/s per core [14]. For on-chip traffic, we follow the AWS Trainium2 technology parameters, defining SBUF to PSUM on-chip bandwidth as 10 TB/s [3].

The second case is a read hit, where a tensor is already resident in either the StRAM or SBUF. In this case a similar equation is used:

$$t_{hop} = t_{read} + \frac{n_{bytes}}{BW_{link}} \tag{4}$$

Our simulator assumes that the memory operations of each core are performed in parallel, and throughout the replay, these latencies are accumulated for each core. DMSim reports the maximum, worst-case latency of all the cores. Currently, the simulator does not model contention delay, inter-core traffic, or compute delay. Therefore, the reported latency values should be interpreted as relative estimates of memory-access-time changes across hierarchy configurations rather than as end-to-end execution latency.

Energy: Energy is calculated similarly, though it encompasses the total energy across all cores. Energy comprises two components, memory access energy and refresh energy.

For each inter-memory hop access, energy is defined as:

$$E_{hop} = (E_{read,per_bit} + E_{write,per_bit}) \cdot n_{bytes} \cdot 8 \quad (5)$$

where E_{read,per_bit} and E_{write,per_bit} are the read and write access energies of the source and target memories, respectively, given in pJ/bit. E_{hop} is the total energy for that memory hop in pJ. Like latency, on-chip read hits accumulate energy without the write access component:

$$E_{hop} = E_{read,per_bit} \cdot n_{bytes} \cdot 8 \quad (6)$$

As for refresh energy, the calculation for each memory is as follows:

$$E_{refresh} = r \cdot n_{occupied} \cdot E_{refresh_per_bit} \cdot 8 \quad (7)$$

where r is the number of refreshes, calculated by taking the total execution time divided by the refresh interval, $n_{occupied}$ is the number of occupied bytes in the memory, and $E_{refresh_per_bit}$ is the refresh energy per bit in pJ. The refresh rate and $E_{refresh}$ per bit are defined in the memory's tech_spec configuration. The total execution time used in the simulation for refresh energy is a conservative estimate, using the baseline execution time for all simulated configurations.

The access energies are charged and accumulate every memory event in the simulator replay. Adding on the refresh energy gives the total simulated energy. In the simulator, interconnect traffic energy is not modeled, as specific parameters to model that energy were not readily available. However, the energy of the traffic is largely tied to the cross-domain traffic, so insights into interconnect energy can be found in that metric.

HBM and Cross-Domain Traffic: HBM traffic accumulates bytes crossing the HBM interface when HBM is the source or destination for interconnect moves. Cross-domain traffic is similarly calculated, though it encompasses all traffic between on-chip and off-chip memories.

IV. Experimental Setup

Tensor profiles were collected for decode passes of Llama-3.2-1B-Instruct and Qwen1.5-MoE-A2.7B [16]–[19]. We ran trace profiling for the above models on a trn2.3xlarge, one Trainium2 device with four logical NeuronCore-v3s. The Neuron Explorer tool collected traces for each core separately. The DMSim ingest function merged the four core decode profiles with tensor execution order mapping and tensor shape classifying heuristics.

To characterize the effects of differentiated memory on Trainium2 hardware, we organized our StRAM and LtRAM area replacement sweeps into three groups, or experiments:

1) StRAM-only: Only StRAM replaced SBUF, leaving the HBM untouched. We swept four points from 10 to 75% SBUF to StRAM area trade using 1T1C eDRAM followed by 3T GC eDRAM. Area replacement was uniform across all cores.
2) LtRAM-only: Only LtRAM replaced HBM, leaving SBUF unchanged. We swept four points from 10 to 75% HBM to LtRAM area trade using RRAM and FeRAM.
3) Full Hierarchy: Both SBUF and HBM were replaced by StRAM and LtRAM, respectively. Four design points with a tiered policy, detailed in Table III, were examined.

Policies and hierarchy configurations for each experiment follow the definitions in Section III.

While the DMSim gathered many metrics through the trace replay, key metrics we considered were cross-domain and HBM access traffic, total energy consumption, and memory traffic latency. Since DMSim does not model compute overlap, HBM traffic is the most reliable simulator output. These values were determined by the methods described in Section III, and the results were extracted from the output JSONs and plotted.

V. Evaluation and Results

This section presents the experimental results and key findings. Llama benefits more strongly from near-core StRAM placement because its traffic is dominated by the KV cache, whereas Qwen benefits more from LtRAM capacity because its traffic is dominated by expert weights. Consequently, the optimal memory-area allocation is model-dependent: near-core-heavy designs provide the largest cross-domain traffic reductions for Llama, while capacity-heavy designs provide the largest HBM traffic reductions for Qwen (Figs. 5–7).

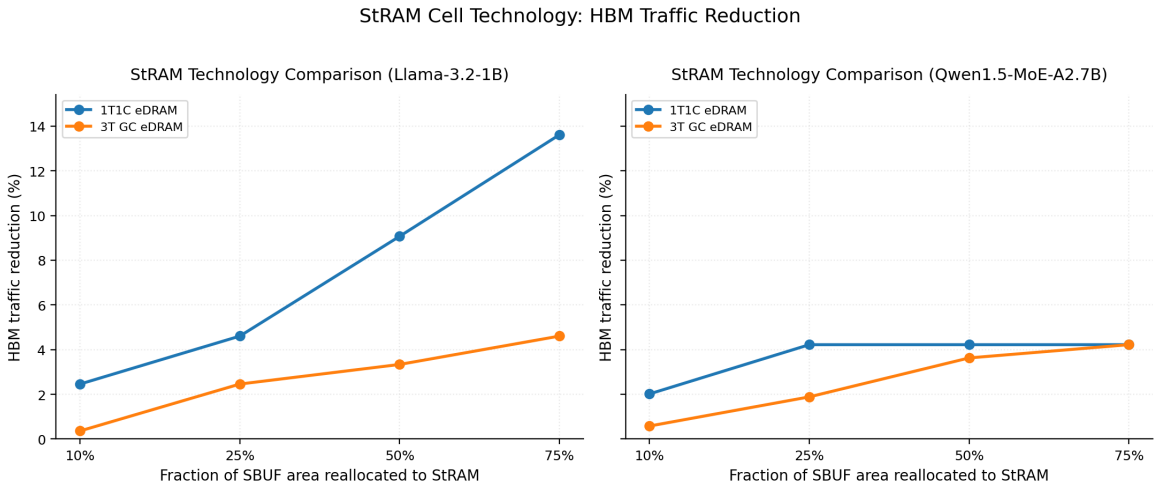
A. StRAM-only (Experiment 1)

Introducing StRAM trades HBM traffic for refresh energy overhead (Fig. 5). On Llama, 1T1C eDRAM at 50% SBUF trade reduced HBM traffic by 9.1%, and at 75% the reduction reached 13.6%. 3T GC eDRAM is substantially weaker, at 3.3% lower HBM traffic at 50% (Fig. 5a). We attribute this result primarily to two effects: on-chip tensor persistence reduced HBM reads, and the higher cell density of 1T1C relative to 3T GC enables more tensors to reside on-chip.

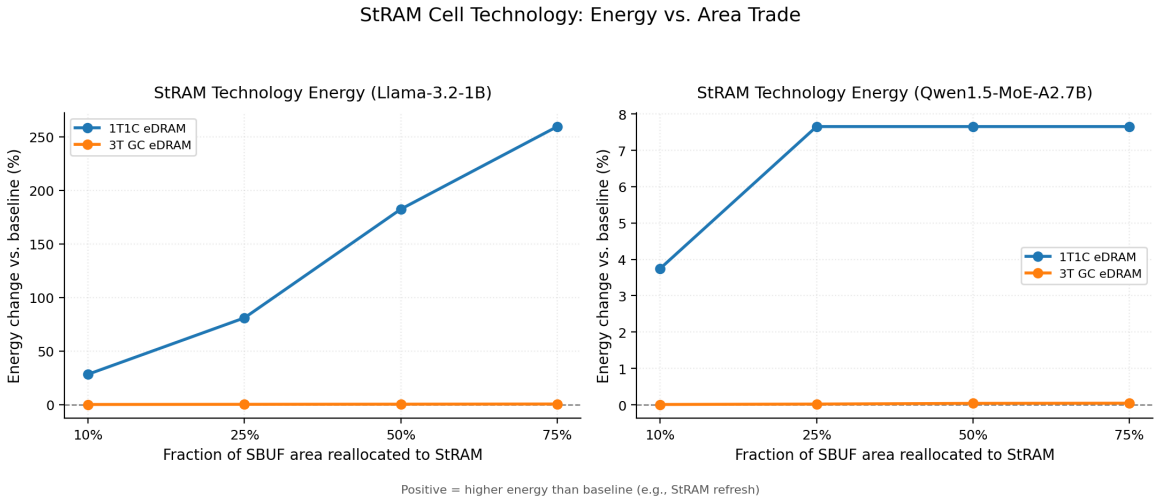
With Qwen, benefits plateau for 1T1C after 25% SBUF replacement, achieving only 4.2% HBM traffic reduction. This pattern reflects the balance of tensor class accesses shown in Fig. 3. In the Qwen model, the weights comprise the majority of tensor movement compared to the KV cache. Paired with the high cell density of 1T1C eDRAM, the entire KV cache is placed into the StRAM after 25% area replacement, and the HBM traffic benefits saturate. Additionally, for Experiment 1 which tests StRAM-only, HBM is the only off-chip memory tier, so all cross-domain traffic can only be HBM traffic.

Takeaway 1. StRAM reduces HBM traffic by keeping more transient, high-reuse tensors on-chip, but benefits are bounded by the size of the target tensor class. Once the entire KV cache fits within the StRAM allocation, further area trades yield no additional HBM traffic reduction.

Fig. 5b illustrates that energy increases for StRAM, primarily due to eDRAM refresh energy. In the Llama model, 1T1C total energy increases sharply (+183% energy at 50% area replacement) due to eDRAM refresh and well-utilized StRAM,



(a)



(b)

Fig. 5: StRAM-only experiment results for Llama and Qwen. (a) Comparison between HBM traffic reduction using 1T1C vs. 3T GC when increasing StRAM area from 10% to 75% for Llama and Qwen models. (b) Comparison between energy change from baseline using 1T1C vs. 3T GC when increasing StRAM area from 10% to 75% for Llama and Qwen.

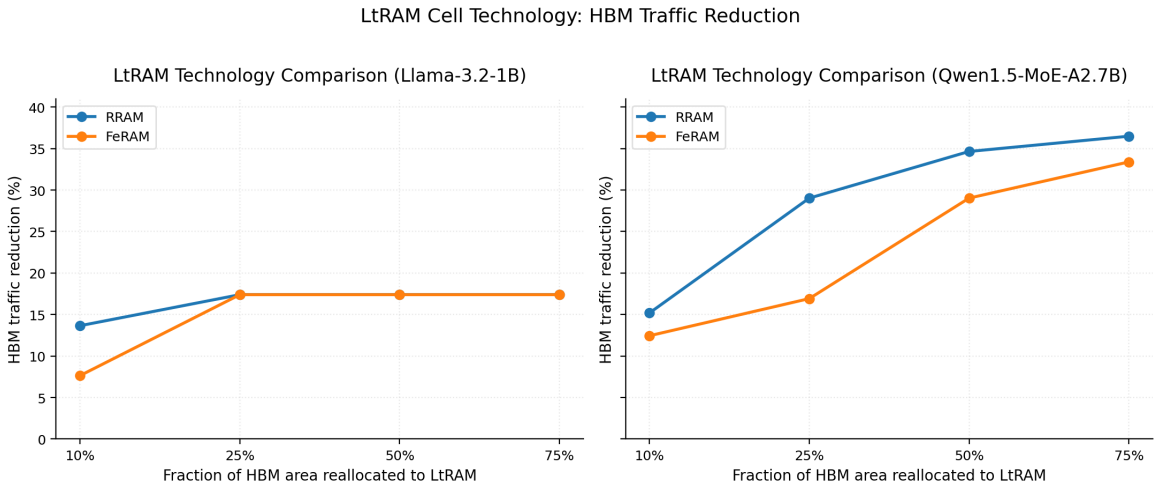
as seen by the HBM traffic reduction (Fig. 5a). However for the Qwen model, the 1T1C energy plateaus similarly to the HBM traffic reduction at 7.7% since the StRAM sees less utilization. Furthermore, the 3T GC eDRAM has lower capacity than 1T1C and also lower refresh energy (Table I), so the change in energy from the baseline compared to 1T1C eDRAM was much smaller (Fig. 5b).

TAKEAWAY 2. 1T1C eDRAM outperforms 3T GC eDRAM in HBM traffic reduction due to its higher cell density, but incurs significantly greater refresh energy overhead. When energy is the binding constraint, 3T GC eDRAM is the preferred StRAM technology.

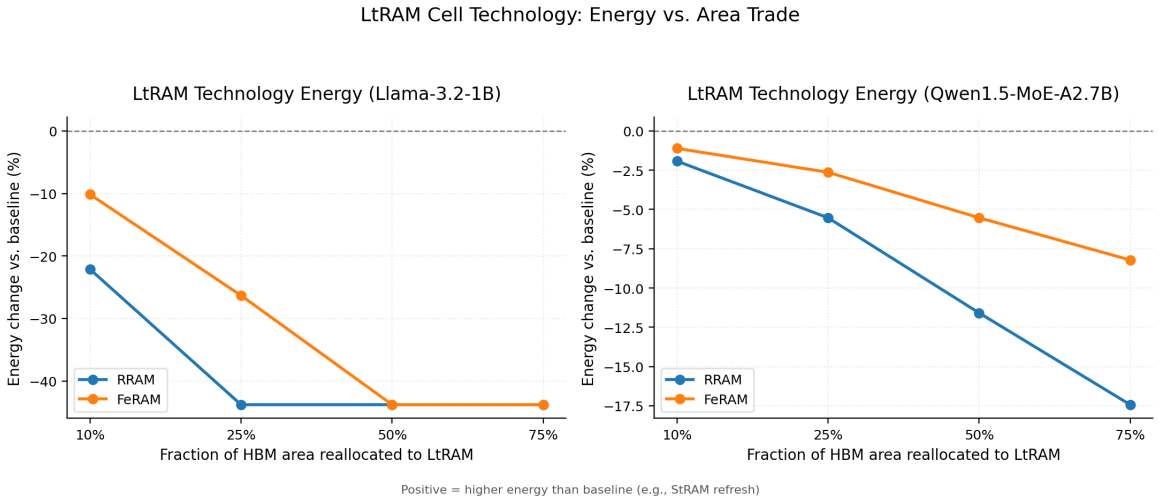
B. LtRAM-only (Experiment 2)

On the Llama model, placing weights in both FeRAM and RRAM LtRAM at 25% HBM area trade reduced HBM traffic by 17.4% (Fig. 6a). Results plateau after 25% because most weights fit into LtRAM, similar to the saturated benefits of the 1T1C StRAM for Qwen (Fig. 5a). However for both Llama and Qwen, RRAM provides greater HBM traffic reduction than FeRAM, likely due to the increased cell density of RRAM (Table I), leading to fewer weight tensors in HBM. Since LtRAM is off-chip, placing weights there reduced HBM traffic without affecting total cross-domain traffic.

TAKEAWAY 3. LtRAM reduces HBM traffic by relocating weight tensors from HBM to an off-chip non-volatile tier, with RRAM achieving greater reductions than FeRAM due to its higher cell density and consequent capacity. Since LtRAM remains off-chip, weight placement in LtRAM reduces HBM traffic without affecting total cross-domain traffic. LtRAM benefits saturate once the weight working



(a)



(b)

Fig. 6: LtRAM-only experiment results for Llama and Qwen. (a) HBM traffic reduction using RRAM vs. FeRAM when increasing LtRAM area from 10% to 75% for Llama and Qwen. (b) Energy change from baseline using RRAM vs. FeRAM when increasing LtRAM area from 10% to 75% for Llama and Qwen.

set fits within the allocated pool, providing a practical LtRAM sizing rule.

Regarding energy in the Llama run, RRAM lowers total energy by 43.8% at the 25% plateau point. At that point, FeRAM reduced total energy by 26.3%, as observed in Fig. 6b.

For Qwen in Fig. 6, benefits continue past 25% HBM area trade because the weight traffic is dominant (Fig. 3), and the pool expert weights exceed the 25% LtRAM pool.

TAKEAWAY 4. LtRAM placement consistently reduces total energy (43.8% reduction for Llama and 17.5% reduction for Qwen), since non-volatile memory carries no refresh penalty and reduces utilization of HBM, the tier with the highest refresh energy.

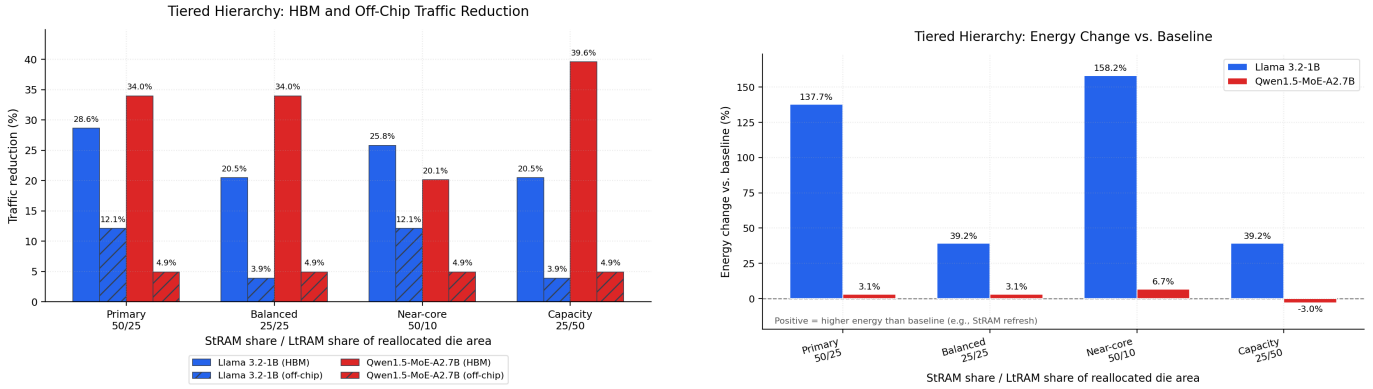
C. Full Hierarchy (Experiment 3)

For the full hierarchy, we chose 1T1C eDRAM for StRAM and RRAM for LtRAM since they demonstrated the greatest reductions in HBM traffic in Experiments 1 and 2. Table IV summarizes the best configuration for each model.

TABLE IV: Best full hierarchy configuration changes compared to baseline. 1T1C eDRAM and RRAM were used as StRAM and LtRAM, respectively.

Model	Best Design	HBM Δ	Cross-Domain Δ	Time Δ
Llama	Primary 50/25	-28.6%	-12.1%	-0.86%
Qwen	Capacity 25/50	-39.6%	-4.9%	-1.95%

On Llama, the greatest HBM reduction achieved was 28.6% using the Primary hierarchy (50/25) (Table IV). Balanced



(a) (b)

Fig. 7: Full hierarchy experiment results for Llama and Qwen. (a) HBM traffic reduction for each hierarchy design for Llama and Qwen. (b) Energy change from baseline for each hierarchy design for Llama and Qwen.

(25/25) and Capacity (25/50) match at 20.5% HBM traffic reduction because 25% LtRAM already holds most weights. Near-Core (50/10) achieves 25.8% HBM traffic reduction. The cross-domain traffic exposes the effect of StRAM on the HBM traffic, with the 50% SBUF traded Near-Core and Primary hierarchies at 12.1% lower cross-domain traffic and the 25% traded Balanced and Capacity at 3.9% (Fig. 7a).

On Qwen, the capacity-heavy hierarchy (25/50) achieved the greatest reduction in HBM traffic of 39.6% compared to Primary and Balanced, which see 34.0% HBM traffic reduction, confirming that MoE workloads need larger LtRAM pools. Near-Core (50/10) underperforms at 20.1% lower HBM traffic with only 10% HBM area traded for LtRAM. Interestingly, the decrease in cross-domain traffic for Qwen is constant at a 4.9% reduction (Fig. 7a). Fig. 5a shows that the HBM traffic benefit of 1T1C eDRAM as StRAM saturates after 25% SBUF area is traded. In the examined mixed hierarchies, the StRAM area trade is always 25% or greater, meaning that StRAM provides the same cross-domain traffic reduction across the four hierarchies. LtRAM contributes changes to the HBM traffic, but not to the cross-domain traffic, as mentioned in the Experiment 2 results discussion.

TAKEAWAY 5. The optimal StRAM/LtRAM area split is workload-dependent. Dense models dominated by KV cache traffic benefit most from near-core StRAM to reduce cross-domain traffic, while sparse MoE models dominated by weight traffic require capacity-heavy LtRAM to capture their larger working set. No single configuration is optimal across model classes.

Regarding the full hierarchy energies seen in Fig. 7b, the Llama results show a significant increase in energy compared to the Qwen results. Even looking at the Balanced hierarchy (25/25), which has the most similar percent change in energy, the Llama model increases energy by 39.2%, whereas Qwen only sees 3.1% increase. Recalling Fig. 5 and 6, the most likely driver for this difference is the StRAM refresh energy, because while LtRAM decreases the energy consumption, the StRAM tends to increase energy consumption through refresh energy. Furthermore, the KV cache-heavy utilization of 1T1C StRAM in the Llama model contributes significant energy consumption, which provides an explanation for the behavior seen in Fig. 7b.

TABLE V: Most Improved Latencies Across Experiments

Experiment	Model	Condition	Time Δ
1 - StRAM-only	Qwen	1T1C @ 25%	-0.27%
2 - LtRAM-only	Qwen	RRAM @ 25%	-1.15%
3 - Full Hierarchy	Qwen	Capacity (25/50)	-1.95%

D. Latency

Table V summarizes the greatest latency improvements across experiments. Results show that latency improvements across all experiments never exceeded 2% latency reduction. This could be due to multiple reasons, though a key factor could be the ratio of HBM reads and writebacks. In the baseline Llama DMSim simulation, SBUF to HBM writebacks constitute a significant portion of memory hops at 134,396 compared to the 4070 HBM to SBUF hops. Because the simulator ties all tensors to their homes at the beginning of the simulation, hierarchy configuration does not affect direct HBM writebacks, so latency improvement is limited. Other factors like placement policy, memory traffic modeling assumptions, etc. could affect the simulated latency improvements, as well.

VI. Discussion and Limitations

A. Placement vs. Trace Labeling

The Neuron Explorer tool separates tensor identities from runtime DMA access events, but lacks model-level semantics. Our tensor mapper is able to infer categories to successfully map most weights and KV tensors, but there are still tensors classified as other because Neuron Explorer's exports use compiler-level names and our mapper cannot reliably link every DMA access to a NEFF slot. These get mapped to HBM

under tiered policy, understating potential gains. Improved DMA-to-semantic attribution would increase policy coverage.

B. Static Placement

Since we focus on the isolated impact of the memory hierarchy changes on decode, we populate memory tiers with their assigned tensors *before* replaying the decode stage trace. Thus, the simulator only records data movement implied by DMA events after $t = 0$. This approach excludes prefill placement and runtime migration that a real system would encounter but is not required for our decode-focused project. Extending the simulator to prefill traces and dynamic placement policies is future work.

C. Analytical Model Fidelity

DMSim is a trace-driven *memory* model, not a full-system performance simulator, and thus makes multiple simplifications. We do not model memory bank conflicts, port contention, or exact NoC routing. Multi-core traces are merged and replayed with per-core SBUF/StRAM state, but inter-core communication is not modeled separately. Reported latency does not account for compute stalls or overlap between memory and compute, and is only the sum of memory access and refresh latencies. These considerations, combined with the latency simulator limitations discussed in Section V, are potential reasons end-to-end latency improvements remain below 2% even when HBM traffic drops substantially.

D. Iso-Area Assumptions

Area trades use literature density figures. Real process integration may differ. Moreover, the collected technology parameters follow different process nodes, and the used technology specifications were a generalized grouping of parameters from a variety of sources, which could lead to inconsistencies with actual technologies. We study single-chip decode only; multi-chip trn2.48xlarge or alternative hardware scaling is left as future work.

E. Endurance

RRAM and FeRAM write endurance and retention are not modeled. For the decode traces we simulate and the policies we consider, these are inconsequential as only read-only weights are placed in LtRAM and the trace duration is orders of magnitude shorter than typical LtRAM retention times. Endurance modeling is left as future work, as other policies may induce writes in the LtRAM, which would bring a need for it.

F. Interconnect Traffic Energy

Only memory access and refresh are considered for the total energy. Dedicated interconnect technology parameters were unavailable for the system we modeled. Since the cross-domain traffic metric is heavily tied to the interconnect traffic energy, changes in cross-domain traffic provide a reasonable representation of interconnect energy changes across different heterogeneous memory hierarchies. Though currently in DMSim, absolute energy totals understate system-wide data-movement energy cost.

G. Generalizability

Results are trace-specific, only modeling the decode stage. Additionally, the simulator only supports Llama-3.2-1B-Instruct and Qwen1.5-MoE-A2.7B. Extending tensor classification to new model architectures requires updating the NeffTensorCatalog heuristics, a targeted engineering task we leave for future work. Finally, the simulator targets AWS Trainium2 and has only been tested on trn2.3xlarge. Support for other Trainium instances or non-Trainium hardware has not been verified.

VII. Conclusion

The results demonstrate that differentiated and heterogeneous memory hierarchies can meaningfully reduce HBM traffic during LLM decode on AWS Trainium2, with energy impact varying by tier. A key finding is that there is no single optimal configuration, and the memory hierarchy design must match the workload characteristics for the greatest benefits. The best full-hierarchy configurations reduce HBM traffic by 28.6% for Llama (Primary) and 39.6% for Qwen (Capacity). The energy results reveal a key asymmetry: adding LtRAM consistently reduces total energy while adding StRAM increases it due to eDRAM refresh overhead, making technology choice as important as area allocation.

These findings carry direct implications for future ML accelerator design. Both hierarchy configuration and technology choice should be treated as high-order architectural decisions, driven by the target model class and system constraints. Dense-model accelerators targeting cross-domain traffic reduction should prioritize StRAM to reduce KV cache traffic, while MoE accelerators would benefit more from bigger LtRAM pools to store large expert weight sets and reduce energy. If a system is energy constrained, 3T GC eDRAM offers the more energy-efficient StRAM option, although 1T1C eDRAM achieves superior HBM traffic reduction.

Future work includes improved tensor mapping, prefill and dynamic-placement traces, refined interconnect energy modeling, and validation against silicon counters.

Acknowledgments

Access to AWS Trainium instances and Neuron Explorer was graciously provided by AWS through the Build on Trainium program.

References

[1] X. Ma and D. Patterson, "Challenges and Research Directions for Large Language Model Inference Hardware," *Computer*, vol. 59, no. 5, pp. 55–64, May 2026, doi: 10.1109/MC.2026.3652916.

[2] Amazon Web Services, "Trainium2 Architecture." *AWS Neuron Documentation*. Accessed: Jun. 8, 2026. [Online]. Available: https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-hardware/trainium2.html.

[3] Amazon Web Services, "The Trainium Memory Hierarchy." *AWS Neuron Documentation*. Accessed: Jun. 23, 2026. [Online]. Available: https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/get-started/about/memory-hierarchy-overview.html.

[4] Amazon Web Services, "Trainium2 Architecture Guide for NKI." *AWS Neuron Documentation*. Accessed: Jun. 23, 2026. [Online]. Available: https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/guides/architecture/trainium2_arch.html.

[5] P. Li *et al.*, "Towards Memory Specialization: A Case for Long-Term and Short-Term RAM," in *Proc. 3rd Workshop on Disruptive Memory Systems (DIMES '25)*, New York, NY, USA, 2025, pp. 27-36, doi: 10.1145/3764862.3768175.

[6] S. Thoziyoor, J. H. Ahn, M. Monchiero, J. B. Brockman, and N. P. Jouppi, "A comprehensive memory modeling tool and its application to the design and analysis of future memory hierarchies," in *Proc. 35th Annu. Int. Symp. Computer Architecture (ISCA)*, Beijing, China, 2008, pp. 51–62, doi: 10.1109/ISCA.2008.16.

[7] K. H. Chiang *et al.*, "Integration of 0.75 V V_{DD} oxide-semiconductor 1T1C memory with advanced logic for an ultra-low-power low-latency cache solution," in *Proc. 2025 Symp. VLSI Technology and Circuits*, Kyoto, Japan, 2025, pp. 1–3, doi: 10.23919/VLSITechnologyandCir65189.2025.11074854.

[8] F. Tu, W. Wu, S. Yin, L. Liu and S. Wei, "RANA: Towards Efficient Neural Acceleration with Refresh-Optimized Embedded DRAM," 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA), Los Angeles, CA, USA, 2018, pp. 340-352, doi: 10.1109/ISCA.2018.00037.

[9] R. Giterman, A. Shalom, A. Burg, A. Fish, and A. Teman, "A 1-Mbit fully logic-compatible 3T gain-cell embedded DRAM in 16-nm FinFET," *IEEE Solid-State Circuits Lett.*, vol. 3, pp. 110–113, 2020, doi: 10.1109/LSSC.2020.3006496.

[10] S. Yu and P.-Y. Chen, "Emerging memory technologies: Recent trends and prospects," *IEEE Solid-State Circuits Mag.*, vol. 8, pp. 43–56, 2016, doi: 10.1109/MSSC.2016.2546199.

[11] S. Salahuddin, K. Ni, and S. Datta, "The era of hyper-scaling in electronics," *Nature Electron.*, vol. 1, 2018, doi: 10.1038/s41928-018-0117-x.

[12] J. Sharda, P.-K. Hsu, and S. Yu, "Accelerator design using 3D stacked capacitorless DRAM for large language models," in *Proc. IEEE 6th Int. Conf. AI Circuits and Systems (AICAS)*, Abu Dhabi, United Arab Emirates, 2024, pp. 487–491, doi: 10.1109/AICAS59952.2024.10595901.

[13] Z. Wang, H. Huang, J. Zhang, and G. Alonso, "Shuhai: Benchmarking high bandwidth memory on FPGAs," in *Proc. IEEE 28th Annu. Int. Symp. Field-Programmable Custom Computing Machines (FCCM)*, Fayetteville, AR, USA, 2020, pp. 111–119, doi: 10.1109/FCCM48280.2020.00024.

[14] Amazon Web Services, "Guideline to Avoid Under-Utilizing DMA Bandwidth." *AWS Neuron Documentation*. Accessed: Jun. 23, 2026. [Online]. Available: https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/deep-dives/nki-dma-bandwidth-guide.html.

[15] Amazon Web Services, "Neuron Explorer." *AWS Neuron Documentation*. Accessed: Jun. 23, 2026. [Online]. Available: https://awsdocs-neuron.readthedocs-hosted.com/en/latest/tools/neuron-explorer/index.html.

[16] Meta, "Llama-3.2-1B-Instruct," *Hugging Face*. Accessed: Jun. 24, 2026. [Online]. Available: https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct.

[17] A. Dubey *et al.*, "The Llama 3 Herd of Models," Nov. 2024. Accessed: Jun. 24, 2026, doi: 10.48550/arXiv.2407.21783. [Online]. Available: https://arxiv.org/abs/2407.21783

[18] Qwen, "Qwen1.5-MoE-A2.7B," *Hugging Face*. Accessed: Jun. 24, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen1.5-MoE-A2.7B.

[19] QwenTeam, "Qwen1.5-MoE: Matching 7B Model Performance with 1/3 Activated Parameters," *Qwen Blog*, Mar. 27, 2024. [Online]. Available: https://qwen.ai/blog?id=qwen-moe